



EVALUACION INDIVIDUAL SOBRE PROYECTO INTEGRADOR

DOCENTE:

JUAN CARLOS TORO LEDEZMA

ESTUDIANTE:

BEKAN DEIVID ZAMBRANA
SUAREZ

GESTION: 2026

MATERIA: DISEÑO WEB II

Contenido

FRAMEWORKS FRONTEND: ANGULAR, REACT Y VUE.JS;Error! Marcador no definido.

¿Qué es un framework frontend?	¡Error! Marcador no definido.
¿Qué es una librería frontend?	¡Error! Marcador no definido.
Problemas que solucionan frente a JavaScript puro ...	¡Error! Marcador no definido.
Concepto de componente	¡Error! Marcador no definido.
Virtual DOM.....	¡Error! Marcador no definido.
Enlace de datos o Data Binding	¡Error! Marcador no definido.
Enlace unidireccional	¡Error! Marcador no definido.
Enlace bidireccional	¡Error! Marcador no definido.
Renderizado del lado del cliente (CSR)	¡Error! Marcador no definido.
Renderizado del lado del servidor (SSR)	¡Error! Marcador no definido.
Diferencia entre CSR y SSR	¡Error! Marcador no definido.
ANGULAR.....	¡Error! Marcador no definido.
Origen	¡Error! Marcador no definido.
Características principales	¡Error! Marcador no definido.
Arquitectura basada en componentes.....	¡Error! Marcador no definido.
Módulos y organización.....	¡Error! Marcador no definido.
TypeScript.....	¡Error! Marcador no definido.
Enlace bidireccional.....	¡Error! Marcador no definido.
Angular CLI.....	¡Error! Marcador no definido.
RxJS	¡Error! Marcador no definido.
Curva de aprendizaje	¡Error! Marcador no definido.
REACT	¡Error! Marcador no definido.
Origen	¡Error! Marcador no definido.
Características principales	¡Error! Marcador no definido.
Componentes	¡Error! Marcador no definido.
JSX.....	¡Error! Marcador no definido.
Virtual DOM.....	¡Error! Marcador no definido.
Flujo de datos unidireccional	¡Error! Marcador no definido.
Gestión del estado	¡Error! Marcador no definido.
React Router	¡Error! Marcador no definido.
Gestión del estado	¡Error! Marcador no definido.
Curva de aprendizaje	¡Error! Marcador no definido.

VUE.JS	¡Error! Marcador no definido.
Origen	¡Error! Marcador no definido.
Características principales	¡Error! Marcador no definido.
Single File Components	¡Error! Marcador no definido.
Reactividad	¡Error! Marcador no definido.
Directivas	¡Error! Marcador no definido.
Vue Router	¡Error! Marcador no definido.
Pinia	¡Error! Marcador no definido.
Vite.....	¡Error! Marcador no definido.
Curva de aprendizaje	¡Error! Marcador no definido.
ANÁLISIS COMPARATIVO.....	¡Error! Marcador no definido.
Cuadro comparativo	¡Error! Marcador no definido.
De manera general.....	¡Error! Marcador no definido.
SELECCIÓN JUSTIFICADA DEL FRAMEWORK PARA EL PROYECTO INTEGRADOR	¡Error! Marcador no definido.
Framework seleccionado para el Proyecto Integrador	¡Error! Marcador no definido.
Justificación	¡Error! Marcador no definido.
Conclusión de la selección	¡Error! Marcador no definido.
CONCLUSIÓN	¡Error! Marcador no definido.
BIBLIOGRAFÍA	¡Error! Marcador no definido.

Dato	Respuesta
Nombre completo	ZAMBRANA SUAREZ BEKAN DEIVID
Grupo / Nombre del proyecto	OMEGA NEXUS
Dominio	Películas / Libros / Música / Juegos
¿Qué partes del proyecto desarrollaste o modificaste?	Frontend / Backend / BD / JavaScript / APIs / JSON / Postman / Otras
Indica archivos, pantallas, endpoints, tablas o funcionalidades en las que participaste	Participe en parte del frontend backend y parte del código de JS.
¿Qué parte conoces menos o no modificaste directamente?	
Menciona una funcionalidad del proyecto que puedas explicar y modificar hoy	En mostrar un dato de la API

5. BLOQUE 1 – QUIZ INDIVIDUAL

A. ARQUITECTURA GENERAL

1. ¿Cómo se comunican normalmente el FrontEnd y el BackEnd?

R. b) El FrontEnd realiza peticiones HTTP al BackEnd y recibe datos, generalmente en JSON.

2. Desarrollo. Explica qué ocurre desde que el usuario hace clic en “Ver detalle de [película/libro/canción/juego]” hasta que la información aparece en pantalla. Menciona JavaScript, petición HTTP, endpoint/PHP, base de datos, JSON y HTML cuando correspondan

R. JavaScript obtiene el identificador del juego y busca la información del catálogo. Los datos son cargados desde la API local mediante una petición HTTP. El backend consulta SQLite y devuelve los datos en JSON. Finalmente, JavaScript procesa la información y actualiza el HTML para mostrar el detalle del juego.

3. Indica qué tecnologías utilizó tu equipo y explica la función concreta que cumple cada una en TU proyecto: PHP, MySQL/PostgreSQL, JavaScript, JSON, API/Endpoints, Framework/Librería y Postman**R.**

- **Python:** funciona como BackEnd y servidor local mediante `server.py`.
- **SQLite:** almacena persistentemente la información de los videojuegos.
- **JavaScript:** controla la lógica, eventos, filtros, CRUD y actualización del HTML.
- **JSON:** permite representar y transportar la información de los videojuegos.
- **API/Endpoints:** permiten la comunicación entre JavaScript y el servidor.
- **Bootstrap:** proporciona componentes, grilla responsive y estilos.
- **jQuery:** facilita la manipulación del DOM y las peticiones AJAX.
- **Postman:** puede utilizarse para probar los endpoints de la API.

B. JAVASCRIPT Y FRONTEND

4. ¿Qué hace el código `fetch(...)`?

R. Realiza una petición al servidor, convierte la respuesta recibida a JSON y posteriormente utiliza los datos para actualizar la interfaz. JavaScript se encarga de controlar esta comunicación y mostrar la información.

5. ¿Qué hace `filter()`?

R. Crea un nuevo arreglo con los elementos que cumplen una condición. En Omega Nexus se utiliza para filtrar videojuegos por criterios como género, plataforma, favoritos o valoración.

6. ¿Qué hace `textContent`?

R. Busca un elemento HTML mediante su identificador y modifica su contenido de texto con la información del videojuego.

7. ¿Qué ocurre si falla una petición AJAX?

R. El error puede detectarse mediante las funciones de manejo de errores de la petición. En Omega Nexus, si falla la carga desde el backend, se conserva el catálogo almacenado en `LocalStorage` como alternativa.

C. BACKEND / PYTHON

8. Menciona un archivo real desarrollado o modificado y explica qué recibe, procesa, consulta y devuelve.

R. El archivo `server.py` funciona como backend de Omega Nexus. Recibe peticiones HTTP de JavaScript, procesa las solicitudes según el endpoint utilizado y consulta o modifica la base de datos SQLite. Finalmente, devuelve respuestas en formato JSON.

Un ejemplo sencillo es:

GET `/api/catalogo`

Este endpoint consulta los videojuegos almacenados en SQLite y devuelve el catálogo en formato JSON.

9. ¿Cuál es la forma adecuada de prevenir una inyección SQL?

R. b) Utilizar sentencias preparadas y parámetros.

D. BASE DE DATOS

10. Describe dos tablas reales del proyecto, indicando claves primarias, claves foráneas y relación.

R.

Tabla juegos:

- Clave primaria: `id`.
- Almacena información principal como título, fecha, valoración, descripción, desarrollador, imagen y URL del juego.
- No posee una clave foránea directa.

Tabla juegos_categorias:

- No posee una clave primaria definida.
- `juego_id` funciona como referencia hacia `juegos.id`.
- `genero` almacena la categoría del videojuego.
- Se relaciona con `juegos` mediante `juego_id`.

La relación permite que un videojuego tenga uno o varios géneros.

11. Explica una consulta SQL real del proyecto.

R. Consulta SQL:

```
SELECT id, api_id, titulo, fecha, valoracion,
descripcion, desarrollador, imagen, juego_url, origen
FROM juegos
ORDER BY titulo
```

Esta consulta obtiene los datos principales de todos los videojuegos almacenados en la tabla `juegos` y los ordena alfabéticamente por título. Se utiliza para construir el catálogo que posteriormente se devuelve al FrontEnd.

E. APIs, JSON Y POSTMAN

12. Menciona un endpoint real: ruta, método HTTP, parámetros, JSON devuelto y uso en el FrontEnd.

R.

Endpoint: `/api/catalogo`

Método: GET

Parámetros: no requiere parámetros.

Respuesta: devuelve un arreglo JSON con los videojuegos del catálogo.

Ejemplo:

```
[
  {
    "id": 1,
    "titulo": "Juego",
    "fecha": "2023-01-01",
    "valoracion": 4.5,
    "generos": ["Acción"],
    "plataformas": ["PC"]
  }
]
```

El FrontEnd muestra esta información en el catálogo.

13. Explica la diferencia entre GET y POST y proporciona un ejemplo real.

R. GET se utiliza para obtener información. En el proyecto, `GET /api/catalogo` obtiene el catálogo.

POST se utiliza para enviar información al servidor. En el proyecto, `POST /api/juego` recibe los datos de un nuevo videojuego y lo registra en SQLite.

F. INTEGRACIÓN

14. Describe el flujo del proyecto.

R.

HTML → JavaScript → API/Backend → Base de Datos → Backend → JSON → JavaScript → HTML

1. **HTML:** presenta la interfaz al usuario.
2. **JavaScript:** detecta las acciones y realiza las peticiones.
3. **Backend:** recibe la petición y determina qué operación realizar.
4. **SQLite:** almacena o proporciona los datos solicitados.
5. **Backend:** procesa los resultados.
6. **JSON:** transporta la información hacia el FrontEnd.
7. **JavaScript:** procesa los datos recibidos.
8. **HTML:** se actualiza para mostrar el resultado.

15. Si al hacer clic en un botón no aparecen resultados, ¿dónde investigaría?

R.

1. **JavaScript:** revisar la consola del navegador para detectar errores.
2. **Network/HTTP:** comprobar si la petición fue enviada y qué respuesta recibió.

3. **Backend:** comprobar que el endpoint esté funcionando y que la consulta a SQLite devuelva información.

6. BLOQUE 2

Opción seleccionada: B – Mostrar un dato adicional del JSON

Mejora propuesta: mostrar el ID de la API

Se utilizará el campo `api_id`, que forma parte de los datos de los videojuegos, pero no se muestra directamente en la interfaz de detalle.

El dato permitirá identificar el videojuego según el identificador utilizado por la API de origen.

Funcionamiento

1. El usuario abre el detalle de un videojuego.
2. JavaScript obtiene la información del juego.
3. El objeto contiene el campo `api_id`.
4. JavaScript toma dicho valor.
5. Se agrega un elemento visual en la ficha del videojuego.
6. El valor de `api_id` se muestra en pantalla.

Ejemplo de modificación

```
$('#detalle-ficha').html(
    '<li>Género(s)<span>' + escaparHTML(textoGeneros(juego)) + '</span></li>' +
    '<li>Plataforma(s)<span>' + escaparHTML(textoPlataformas(juego)) + '</span></li>'
    +
    '<li>Desarrollador<span>' + escaparHTML(juego.desarrollador) + '</span></li>' +
    '<li>Lanzamiento<span>' + formatearFecha(juego.fecha) + '</span></li>' +
    '<li>Origen<span>' + (juego.origen === 'api' ? 'API externa' : 'Catálogo local') +
    '</span></li>' +
    '<li>ID de API<span>' + escaparHTML(juego.api_id || 'No disponible') +
    '</span></li>'
);
```

El valor no será escrito manualmente, sino que será obtenido directamente del objeto JSON recibido.

Se realiza esta modificación porque no requiere crear un nuevo endpoint ni modificar la estructura de la base de datos. Solamente se aprovecha un dato que ya forma parte de la información del videojuego y se incorpora a la interfaz mediante JavaScript.